Corrigé – Évaluation de NSI

Réseaux, récursivité et programmation dynamique

Exercice 1 – Réseaux

1. Adressage IPv4

a. 11000000.10101000.10000000.10000011 correspond à **192.168.128.131**.

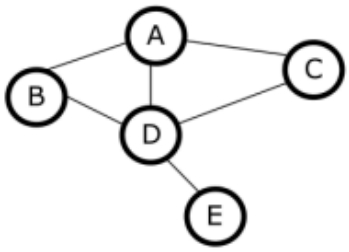
b. Le dernier octet est codé sur 8 bits, donc il existe **256** valeurs possibles, de 0 à 255.
Dans un réseau classique, seules **254 adresses** sont en général attribuables à des machines, car on retire l'adresse réseau et l'adresse de broadcast.

2. Protocole RIP

a. Les routeurs directement reliés à A sont ceux de métrique 1 depuis A : **B, C et D**.

b. Représentation simplifiée du réseau :

1.
   a. 192.168.128.131
   b. nombre d'adresses possible 256 (nombre de machines réellement adressables : 254, car il faut retirer le 0 (adresse réseau) et le 255 (adresse de broadcast)).
2.
   a. liste des routeurs directement reliés à A (métrique = 1) : B, C et D
   b.



3.

Débit	100 kbps	500 kbps	10 Mbps	100 Mbps
Métrique associée	1000	200	10	1

3. Protocole OSPF

Débit	100 kbps	500 kbps	10 Mbps	100 Mbps
Métrique associée	1000	200	10	1

4. Routage avec OSPF

a. Le chemin de coût minimal pour aller de F à I est :
F → H → J → K → I, de coût total **13**.

b. Table complétée du routeur F :

4.

a. le chemin emprunté par un message pour aller de F à I est :
F -> H -> J -> K -> I (coût de 13, tous les autres chemins possibles ont un coût supérieur à 13).

b.

Destination	Métrique
F	0
G	8
H	5
I	13
J	6
K	8
L	11

c.
Trois routeurs permettent d'accéder à F directement I, H, et G. Si H tombe en panne, il resterait donc uniquement G et I. Or le coût de la liaison I -> F est de 20, ce qui rend préférable l'option de passer par G dans tous les cas de figure (par exemple, même pour aller de I à F, le chemin retenu serait I -> K -> J -> G -> F avec un coût de 19).

c. Une panne du routeur **H** suffit. Dans ce cas, les accès directs vers F depuis les autres réseaux passent principalement par **G**, car l'autre possibilité via I puis F est plus coûteuse.

Exercice 2 – Récursivité et programmation dynamique

Question 1

Pour une barre de 4 mètres, on peut obtenir :

- 1 barre de 4 m : 9 euros
- 1 barre de 3 m et 1 barre de 1 m : 8 + 1 = 9 euros
- 2 barres de 2 m : 5 + 5 = 10 euros
- 1 barre de 2 m et 2 barres de 1 m : 5 + 1 + 1 = 7 euros
- 4 barres de 1 m : 4 × 1 = 4 euros

Le **revenu optimal** est donc **10 euros**.

Question 2

Fonction récursive complétée :

```python
def revenu_barre(n):
    # Les indices de prix représentent la longueur
    # de la barre.
    prix = [0,1,5,8,9,10,17,17,20,24,30]

    # Cas d'arrêt
    if n == 0:
        return 0

    # Cas général
    else :
        revenu = 0
        for i in range(1, n+1) :
            revenu = max(revenu, prix[i] + revenu_barre(n-i))

        return revenu
```

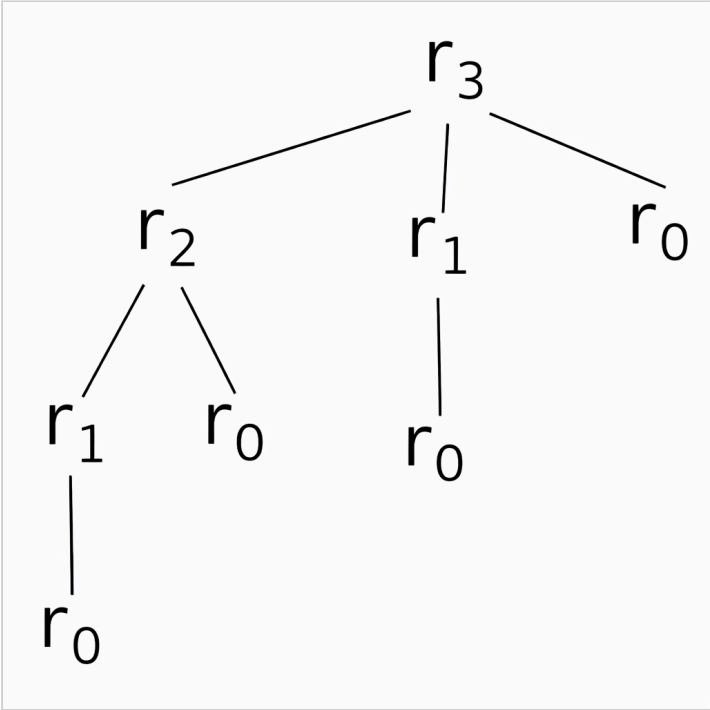
Question 3

Il suffit de modifier la liste `prix` pour utiliser les nouveaux tarifs :

```
prix = [0, 1, 6, 9, 11, 12, 19, 20, 23, 24, 26]
```

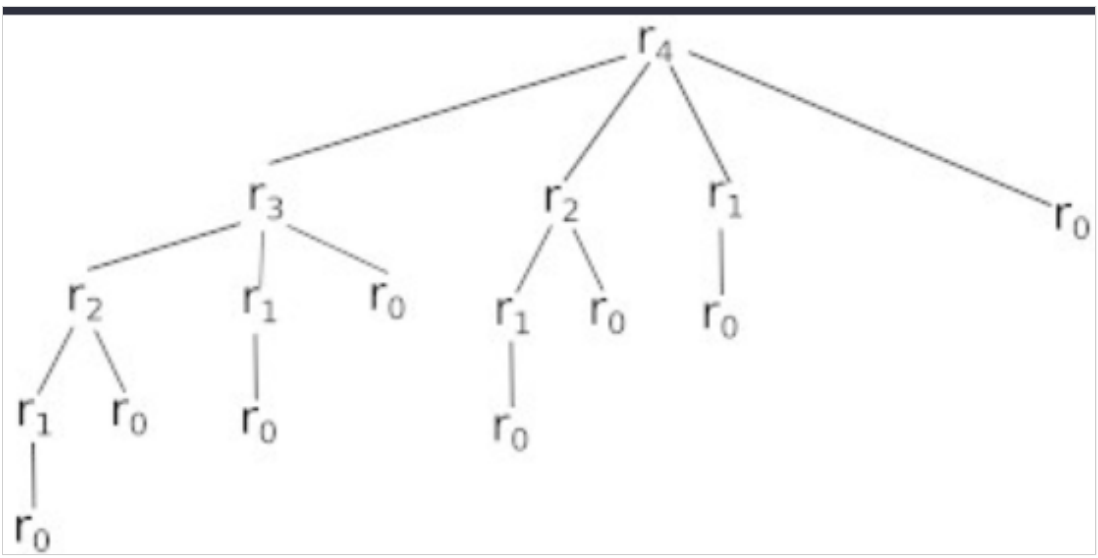
Question 4

Voici l'arbre d'appels récursifs du calcul de `r(3)` :



1. La valeur `r(1)` est calculée **2 fois**.
Cela provoque des **recalculs inutiles** et donc une **complexité exponentielle** lorsque `n` grandit.

2. Arbre d'appels récursifs de `r(4)` :



Question 5

Version descendante avec mémoïsation :

Version avec paramètre memo

```python
# Les indices de prix représentent la longueur de la barre
prix = [0, 1, 5, 8, 9, 10, 17, 17, 20, 24, 30]


def revenu_barre(n, memo = None):
    # Création du dictionnaire (premier appel)
    if memo == None :
        memo = {}

    # Cas d'arrêt
    if n == 0:
        return 0

    # Cas général avec mémoïsation
    if n not in memo:
        memo[n] = 0
        for i in range(1, n+1):
            memo[n] = max(memo[n], prix[i] + revenu_barre(n - i, memo))

    return memo[n]


# Test
print(revenu_barre(4))  # attendu : 10
```

```
10
```

Version avec variable globale

```python
# Initialisation du dictionnaire
memo = {}

# Les indices de prix représentent la longueur de la barre
prix = [0, 1, 5, 8, 9, 10, 17, 17, 20, 24, 30]


def revenu_barre(n):
    # Cas d'arrêt
    if n == 0:
        return 0

    # Cas général avec mémoïsation
    if n not in memo:
        memo[n] = 0
        for i in range(1, n+1):
            memo[n] = max(memo[n], prix[i] + revenu_barre(n -

    return memo[n]


# Test
print(revenu_barre(4))  # attendu : 10
```

```
10
```

Remarque : en Python, pour que la mémorisation soit pleinement efficace, le dictionnaire `memo` doit être partagé entre les appels récursifs.

Question 6

Version ascendante non récursive :

```
def revenu_barre(n):
    # Les indices de prix représentent la longueur
    # de la barre.
    prix = [0,1,5,8,9,10,17,17,20,24,30]
    revenu = [0]*11

    for i in range(1, n+1) :
        for j in range(i+1) :
            revenu[i] = max(revenu[i], prix[j] + revenu[i-j])

    return revenu[n]
```